

Precision Timing & Skill Check

*Version : **1.0***
***Developed by:** [@sazoncomcafe]*
***Last Updated:** October 10, 2026*

Introduction **3**
Objective **3**
How to Install This Resource **3**
"What will I be able to create?" **4**
About Textures 5
Industry Inspirations & Examples 5
Structure **6**
Packed Scenes 6
Individual Nodes 7
Prefabs **7**
Creating Examples from Scratch **8**
Examples 1 and 2 8
Building Example 3 8
SingletonSkillCheck (AutoLoad) **18**
Signals 18
SkillCheckController **19**
Inspector 19
Methods 19
_ready() 19
play_skill_check(): 19
pause_skill_check(): 20
Methods Connected to SingletonSkillCheck AutoLoad Signals 20
spawn_my_skill_check(_scene_skill_check : PackedScene): 20
get_hit_zones_count(): 20

Skill Check **21**
Inspector 21
Methods 21
_physics_process() 21
_play_or_pause(play : bool) 22
get_hit_zone_count() 22
Manager HitZones **22**
Methods 23
_ready() 23
organize_hit_zones() 24
update() 24
PathSkillCheck **25**
Inspector 25
Methods 25
rules_build_path() 25
get_size_path() -> float 26
get_hit_zones()-> Array: 26
get_pointer() -> Pointer 26
add_hit_zones() 26
Signal 26
new_hit_zones 26
BarPathSkillCheck 27
Methods 27
CirclePathSkillCheck 28
Inspector 28
Methods 28
ArcPathSkillCheck 29
Inspector 29
Methods 30
HitZone **30**
Inspector 31
Methods 32
get_width_texture() -> float 32
get_hit_area() -> Array 32
AreaHitZone 32
Inspector 32
Methods 32
get_type()->String: 32
get_size_shape(): 32
hittable(): 33
Pointer **33**
Inspector 33
Methods 34
_ready(): 34

_physics_process(delta) and move_pointer(delta)....35
_is_hit_zone(area : Area2D)....35
_can_hit(area : Area2D)....36
not_hit(area : Area2D)....36
getters....36
ResourcePointer....36
License....37
Conclusion....37

Introduction

Inspired by a wide variety of games, both Indie and AAA, featuring familiar mechanics. Easy to work with in Godot Engine version 4.x onwards, having been developed in Godot 4.6.2 stable. Here you will find everything you need to manage and integrate this feature, making it simple to create your own minigame for your game.

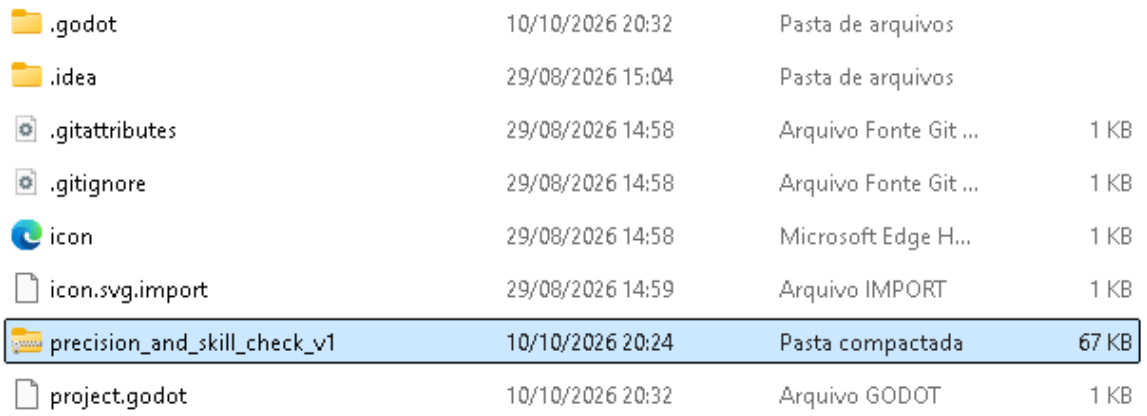
This feature aims to follow SOLID principles and the Observer Architecture pattern as closely as possible.

Objective

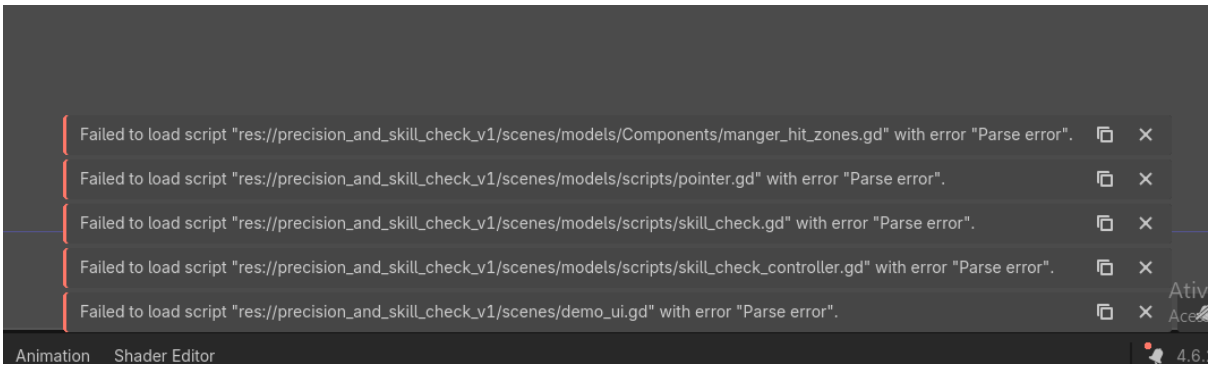
The objective of this Precision and Skill Check minigame resource is to provide a fully customizable foundation that is easy to implement and customize, with all the underlying logic already abstracted, featuring options directly through the Engine's Inspector and built-in documentation tooltips when hovering over certain parameters.

How to Install This Resource

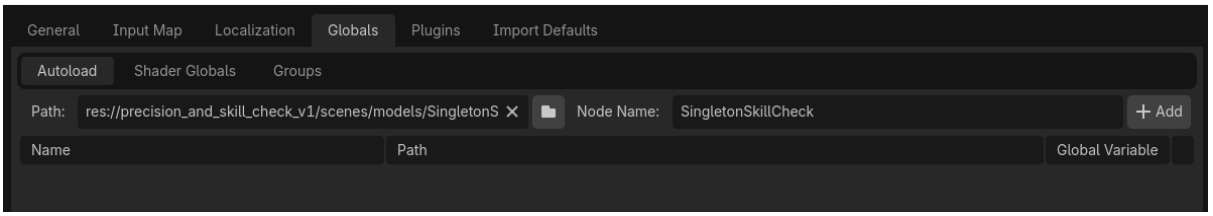
1- Extract the ZIP file inside your project.



***WARNING*: When opening Godot, several errors will appear. They are normal because we still need to configure the Singleton.**



2- In Project Settings, under the Globals tab, select the SingletonSkillCheck file, give it the exact same name, and click **ADD**



3- Now you are ready to use it.

"What will I be able to create?"

With this resource, you will be able to create minigames similar (though not identical) to those found in various types of games.

ALERT: Since the goal of this resource was to remain simple and easy to use, the main focus was on abstraction. Therefore, audio and animations are up to the user to implement.

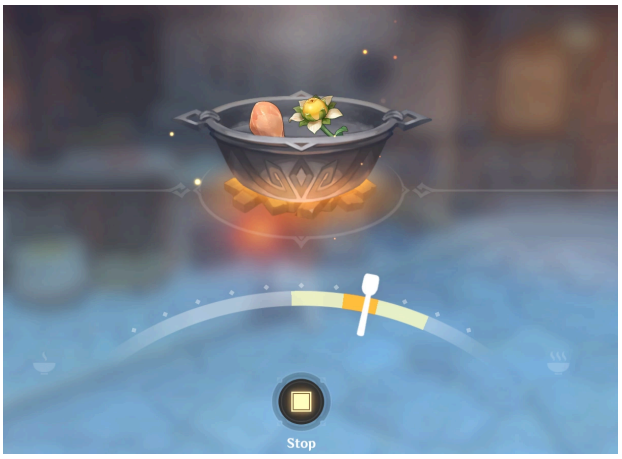
About Textures

ou will notice that in many nodes, the "Texture" attribute is a Sprite2D. Feel free to change it. The goal was to keep this asset as simple and customizable as possible, abstracting all core logic so developers can focus on design or scoring systems.

Industry Inspirations & Examples



Dead by Daylight - 2016



Genshin Impact - 2020



Video Horror Society (Game)



Halloween The Game - 2026

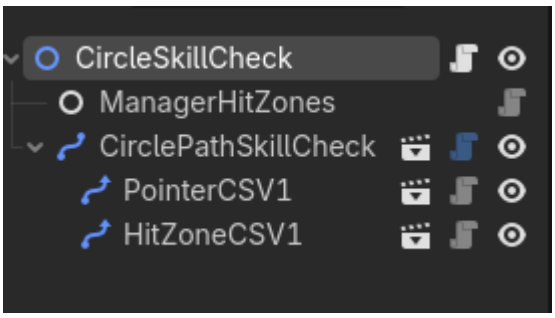


Choo-Choo Charles - 2022

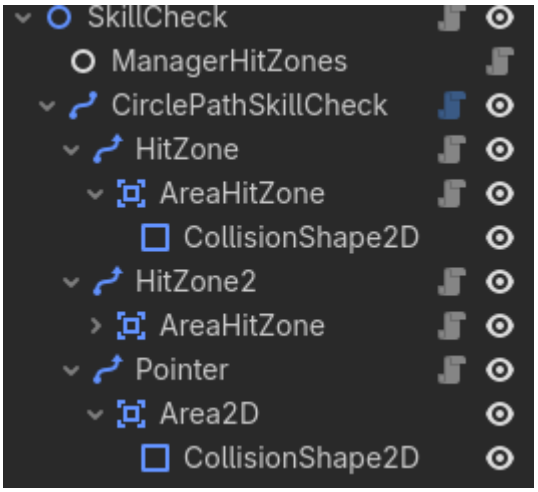
Structure

Packed Scenes

Check out the `circle_skill_check.tscn` file.

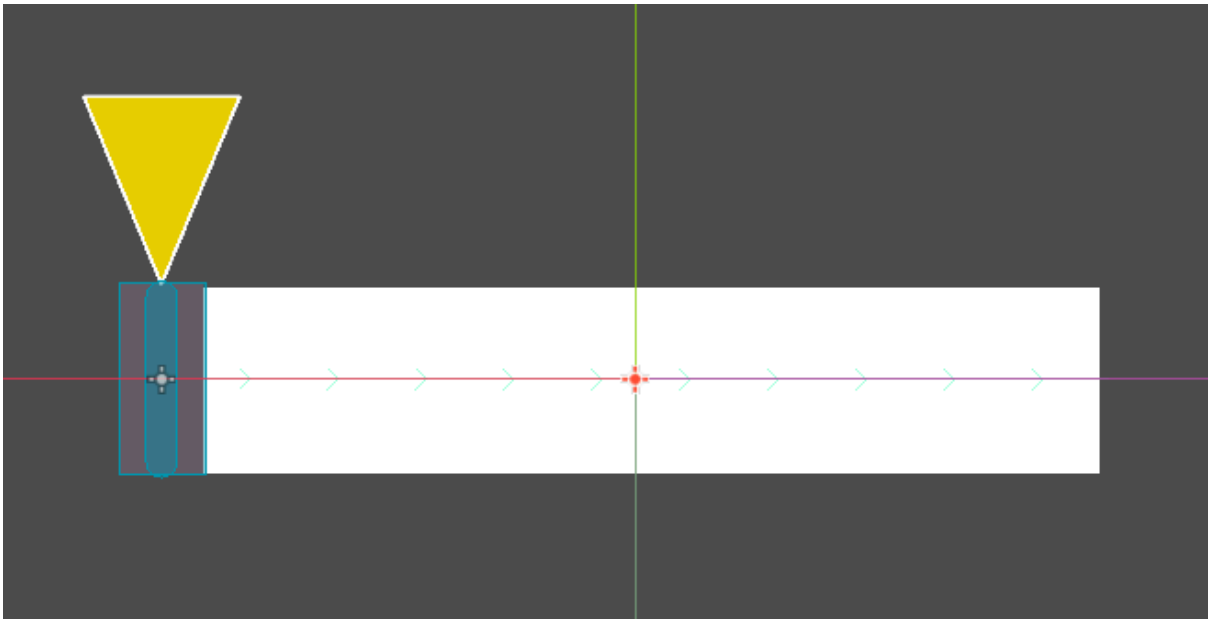


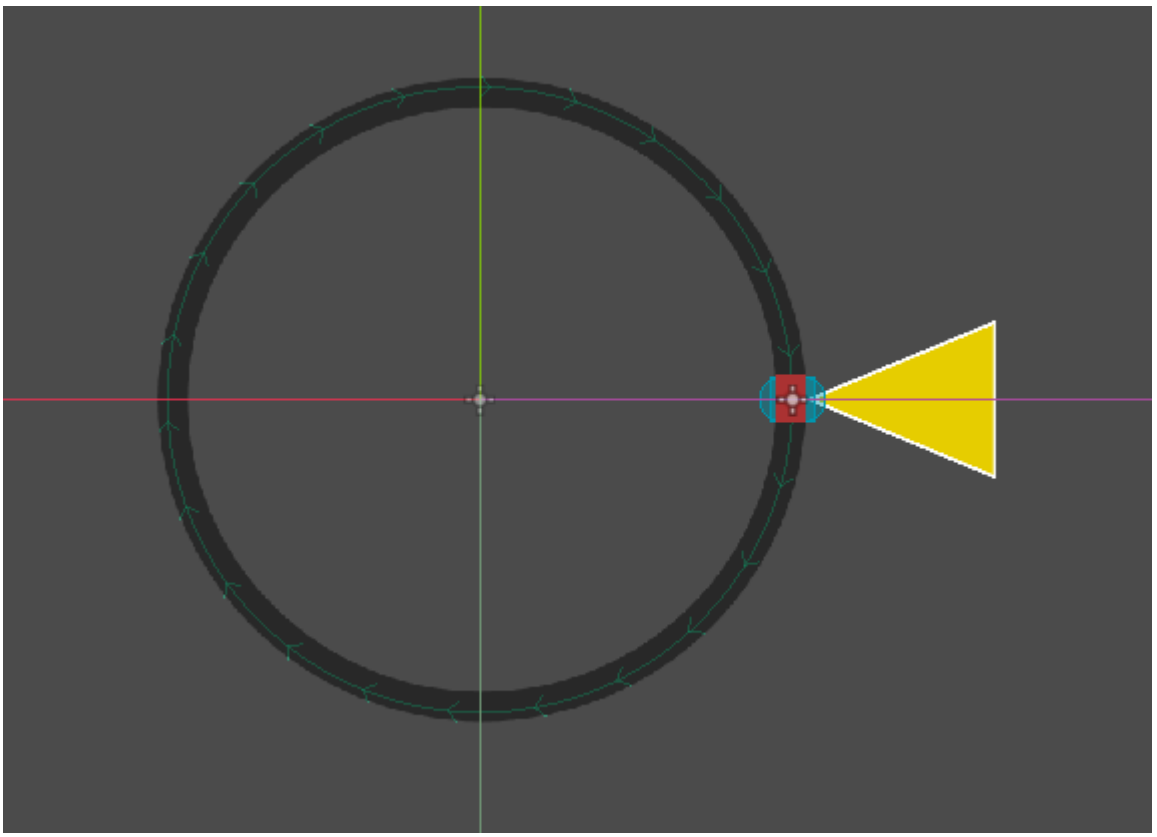
Individual Nodes



***NOTE**: Textures and audio are omitted.*

Prefabs





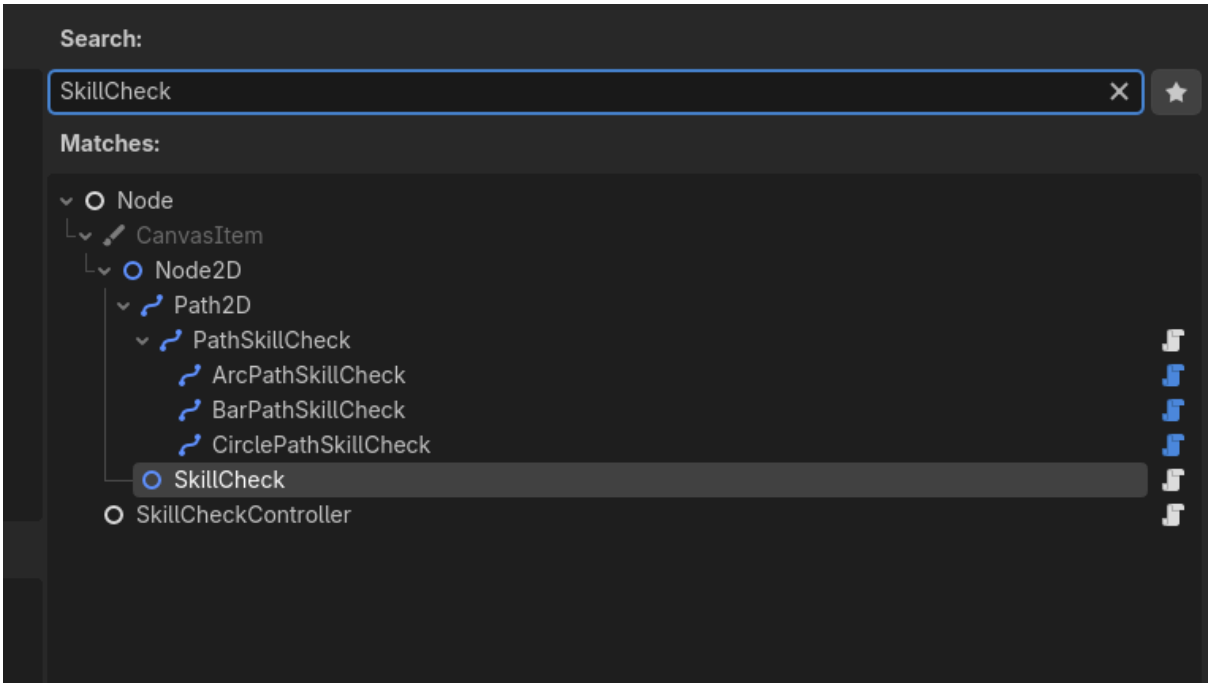
Creating Examples from Scratch

Examples 1 and 2

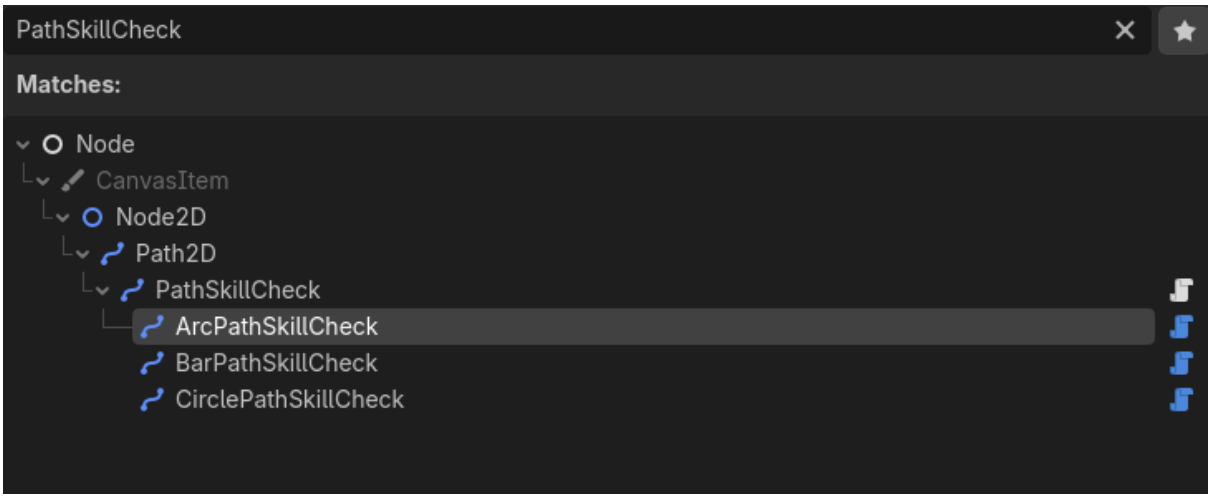
Clicking here allows you to watch the step-by-step creation of Examples 1 and 2: : https://youtu.be/012sXF4XGPo

Building Example 3

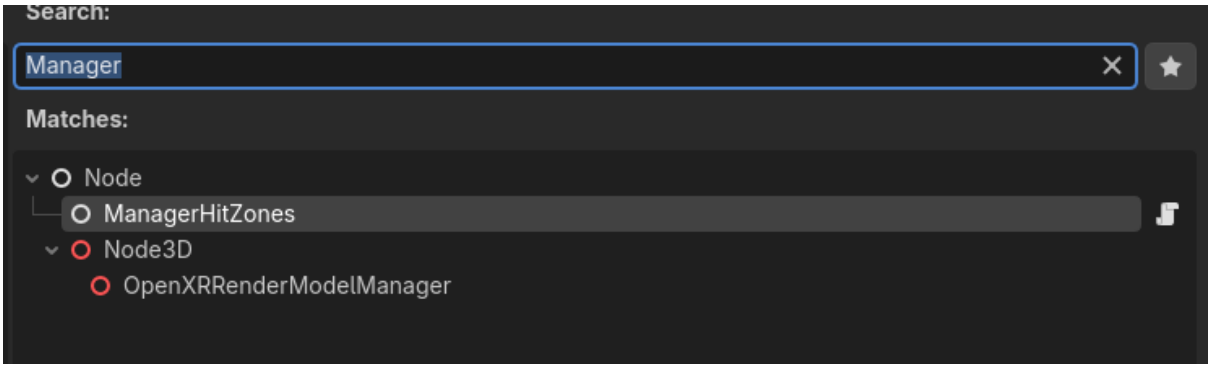
1- Add a Node of the `SkillCheck` class.



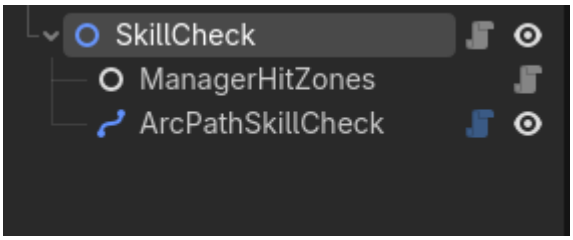
2-Add a `PathSkillCheck` Node as a child of `SkillCheck`. In this case, we will add `ArcPathSkillCheck`. (Note: You can check the specifics of each class in the rest of the documentation).



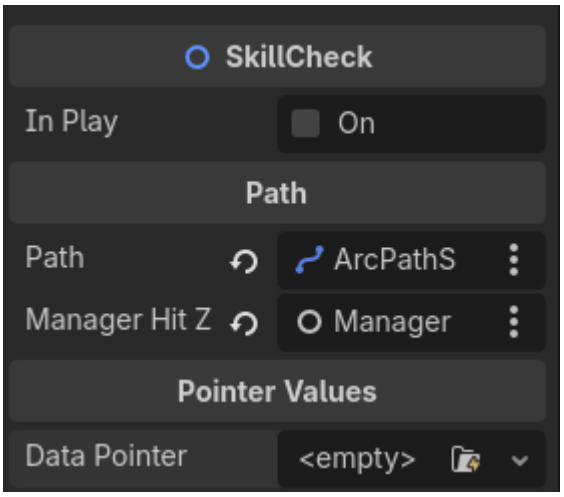
3- Also add a `Manager HitZone` Node (it can be a child of `ArcPathSkillCheck` or placed elsewhere in the hierarchy).



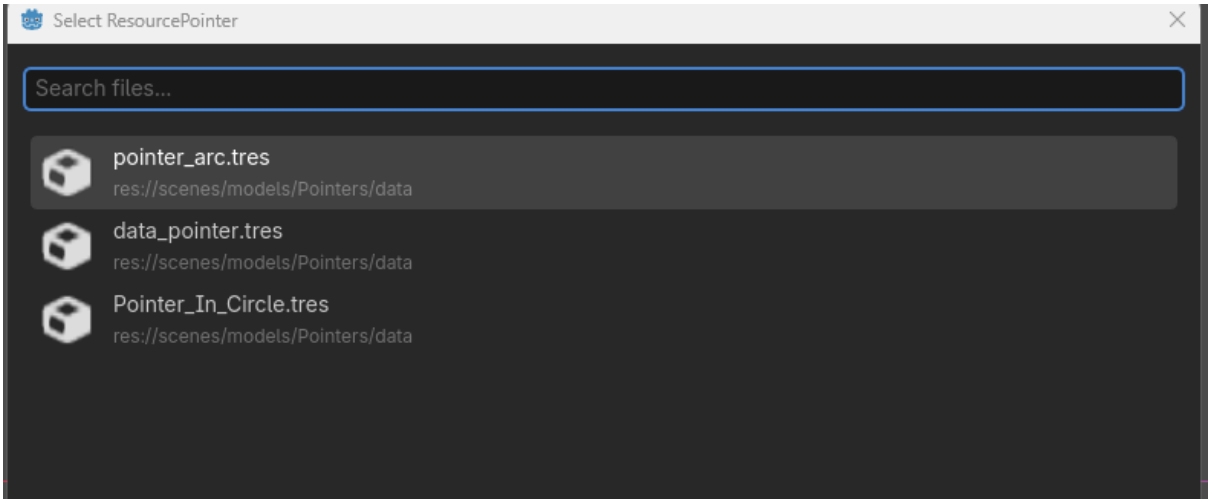
3.5. Your node structure should look somewhat like this:



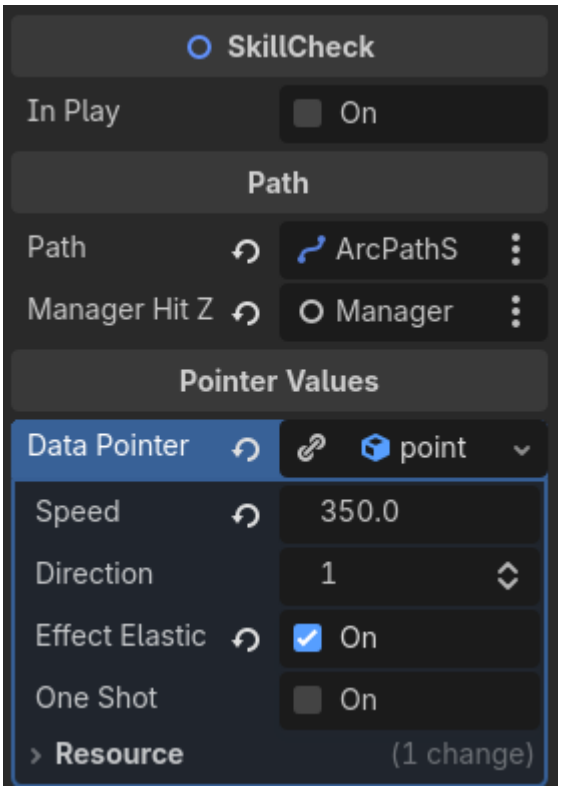
4- With `SkillCheck` selected, assign both the `Path` and the `Manager HitZone` in the Inspector to their respective Nodes.



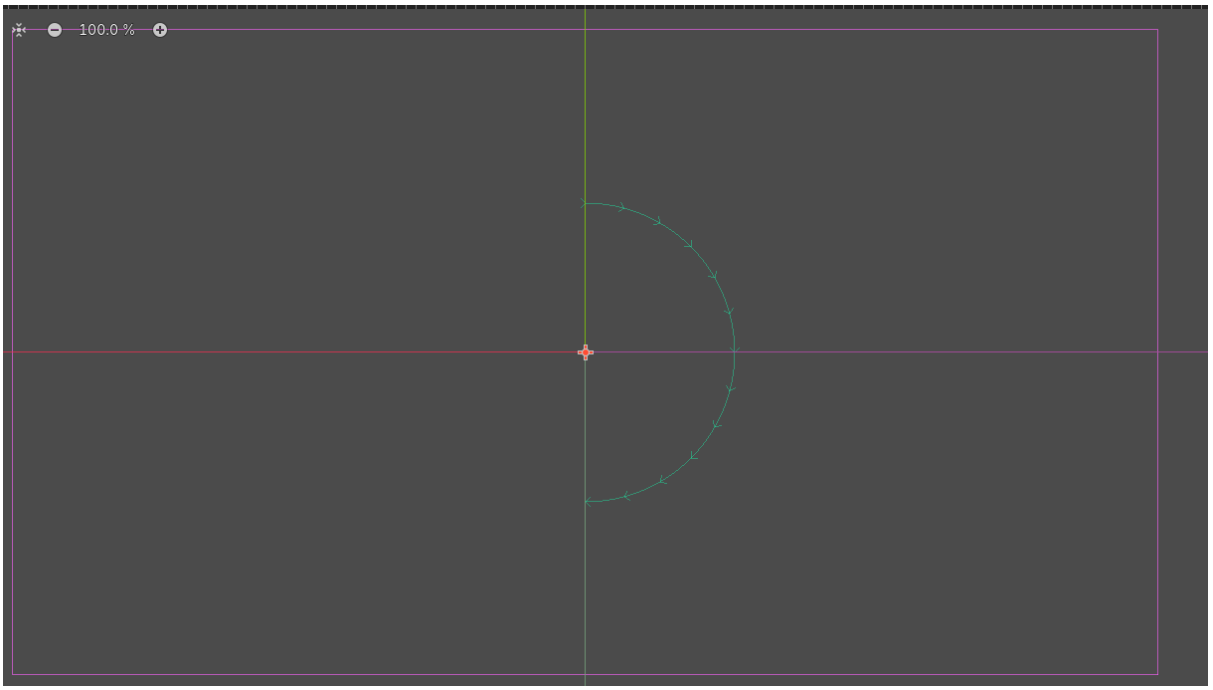
5-In Data Pointer, select the pointer/selector settings. In this case, we will use the prefab resource `pointer_arc.tres`.

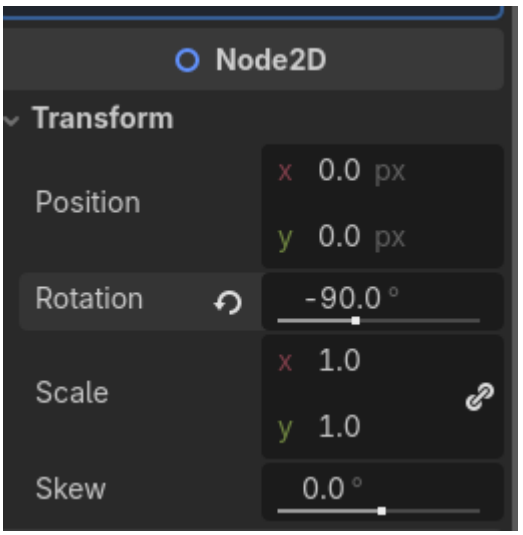


5.5- Your `SkillCheck` inspector should look similar to this:

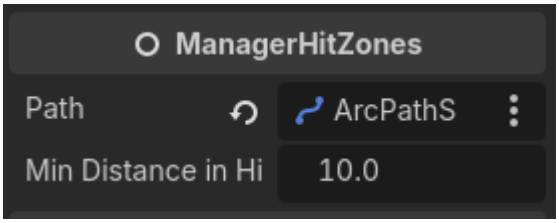


6- In the Inspector, your Path will appear as shown. To adjust its orientation, go to the `Transform` property of the `Node2D` and rotate it -90 degrees.

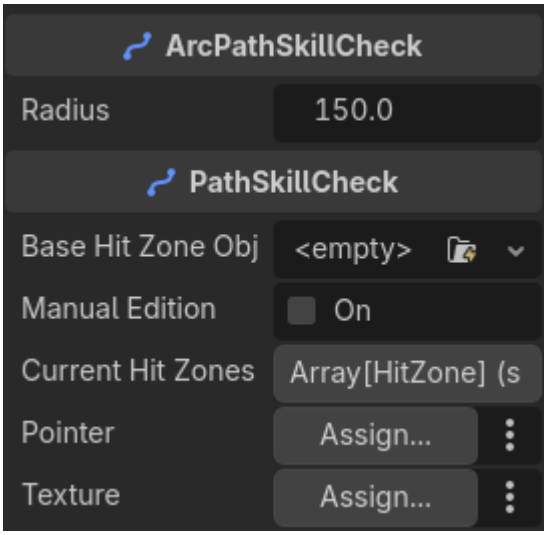




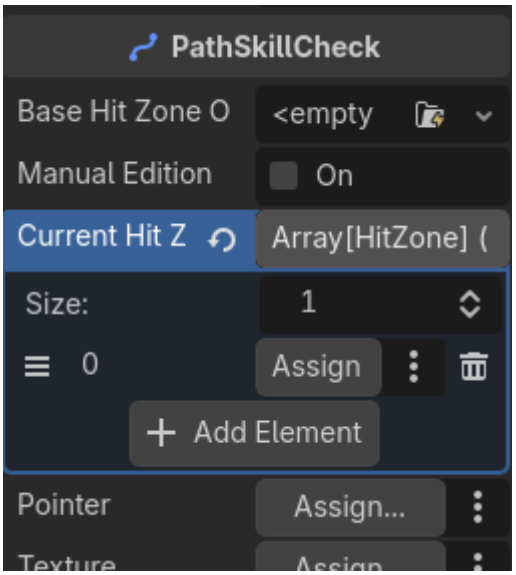
7-Select `ManagerHitZones` and assign `ArcPathSkillCheck` in its Inspector. It should look like this:



8- Select `ArcPathSkillCheck` and review its Inspector settings: .



Radius: Adjusts the radius/size of the arc to be drawn.
Base HitZone: Assign a `PackedScene` here to spawn dynamic HitZones.
Manual Edition: Enable if you wish to modify the path using standard `Path2D` tools.
Current Hitzone: Lists the current HitZones on your Path. (To learn more, search for "HitZones" in this documentation). Here we start by adding an element array slot and assigning a Node later.
Pointer: The pointer/selector node that we will also create.



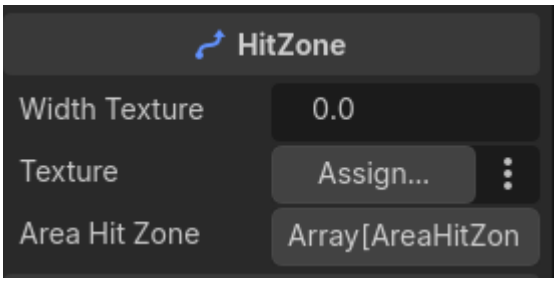
8-Add a `HitZone` Node as a child of `ArcPathSkillCheck`.



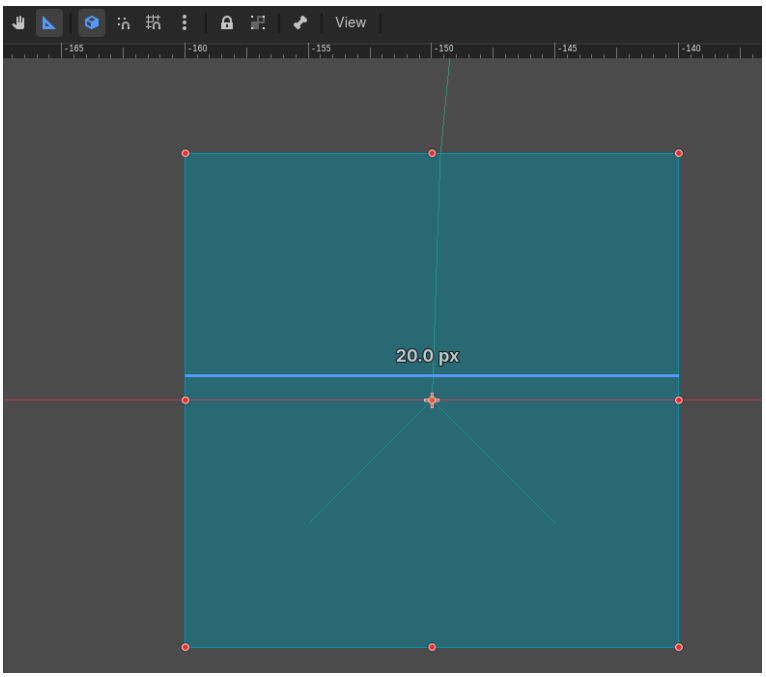
9-Add an `AreaHitZone` Node as a child of the `HitZone` Node (since it inherits from `Area2D`, it requires a `CollisionShape2D`



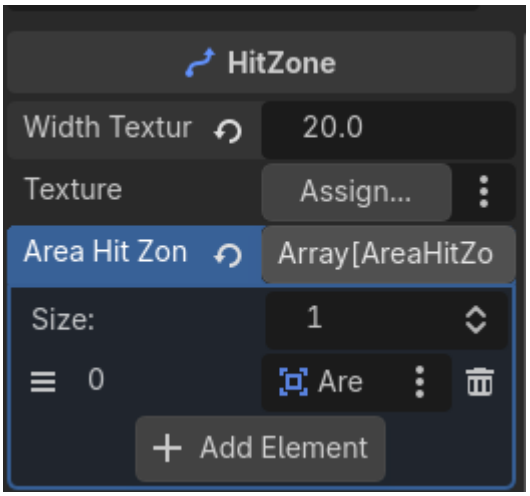
10- Select `HitZone` in the Inspector:



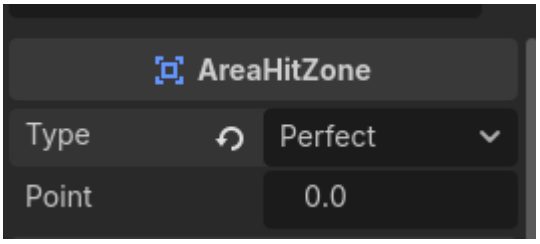
Width Texture: Enter the pixel width of the Area/Texture (for more details, check the "HitZone" section in this documentation). Since we are not using textures here, enter the width matching the `CollisionShape` of your `AreaHitZone`.



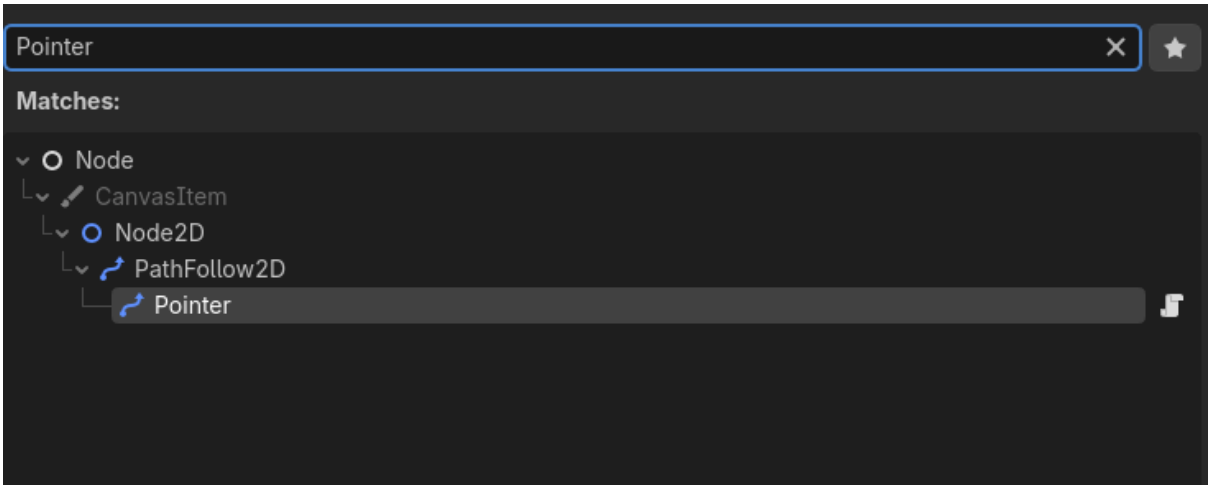
Area Hit Zone: Add 1 element array slot and assign the child `AreaHitZone` Node.



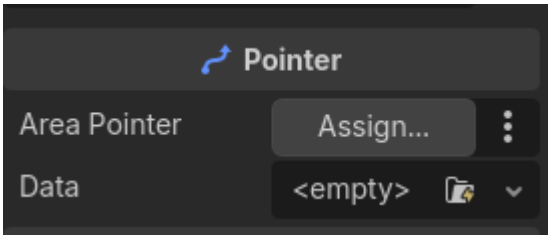
11-Select `AreaHitZone` and set its hit/scoring type in the Inspector



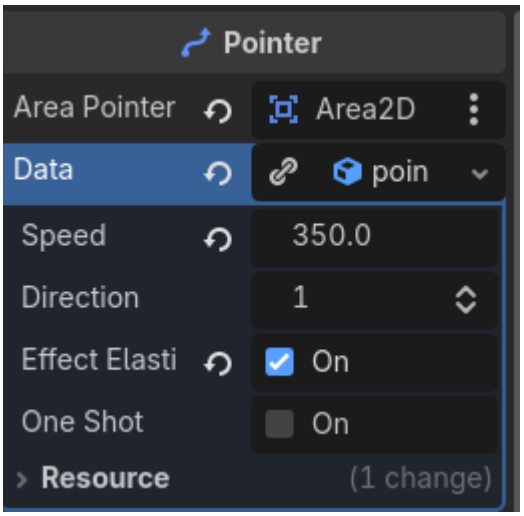
12- Select `ArcPathSkillCheck` and add a `Pointer` Node as its child.



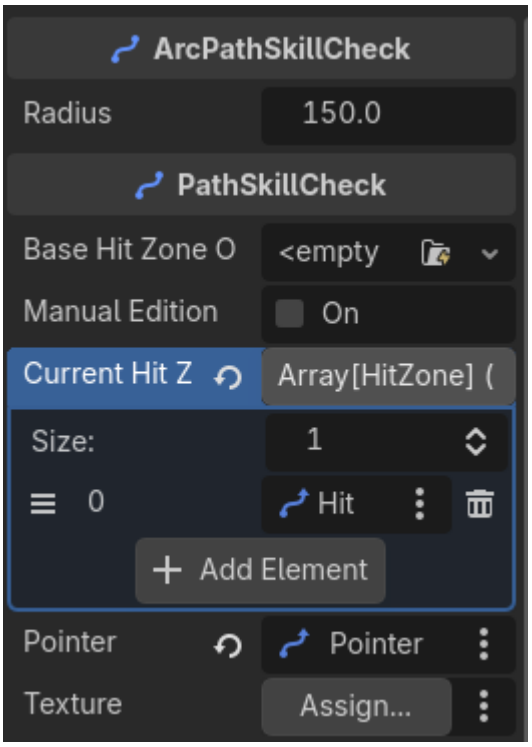
13- Notice the `Pointer` Inspector properties:

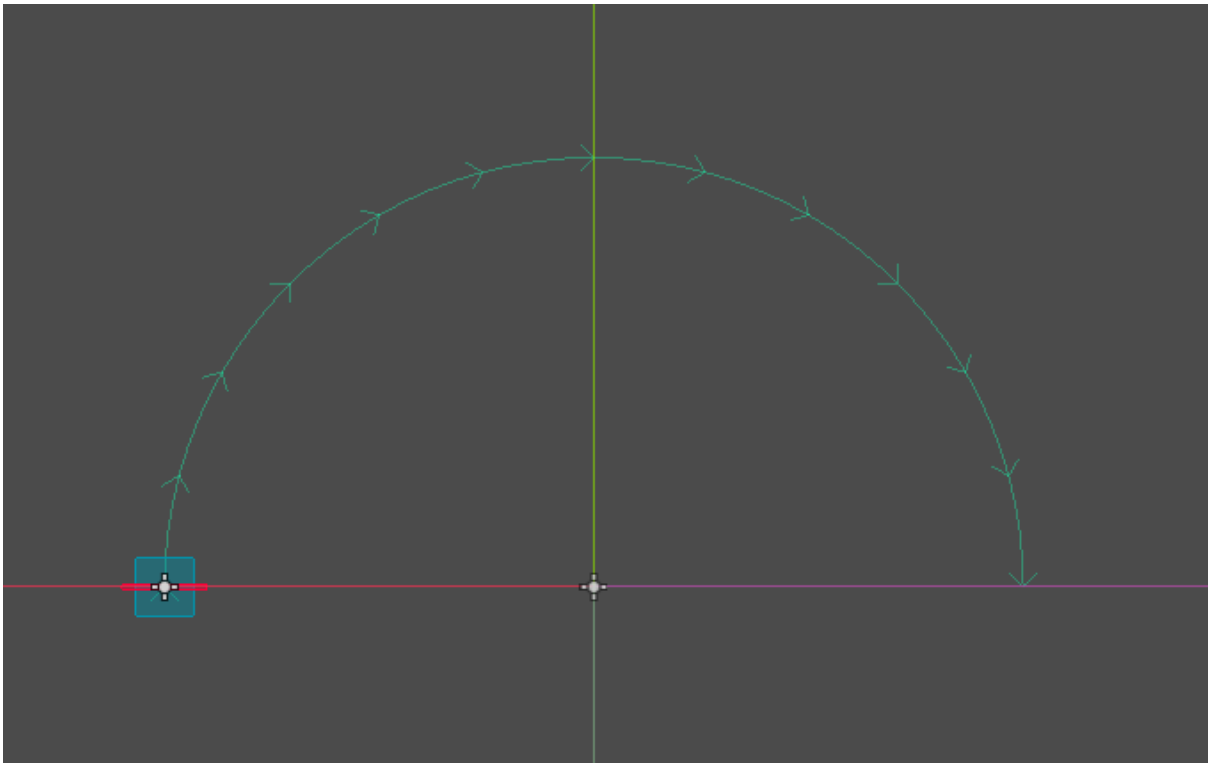


AreaPointer: A standard `Area2D` (a cylindrical/circle collision shape was used for this example).
Data: The pointer traits/settings — select the same resource file chosen in `SkillCheck` (in this case: `pointer_arc.tres`).

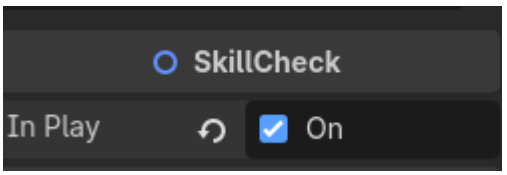


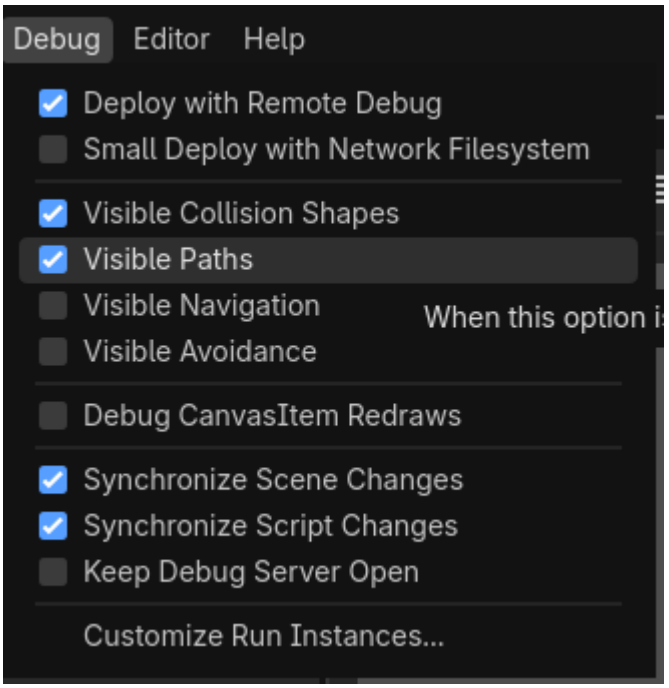
14-Go back to `ArcPathSkillCheck` and assign the `HitZone` and `Pointer` references.



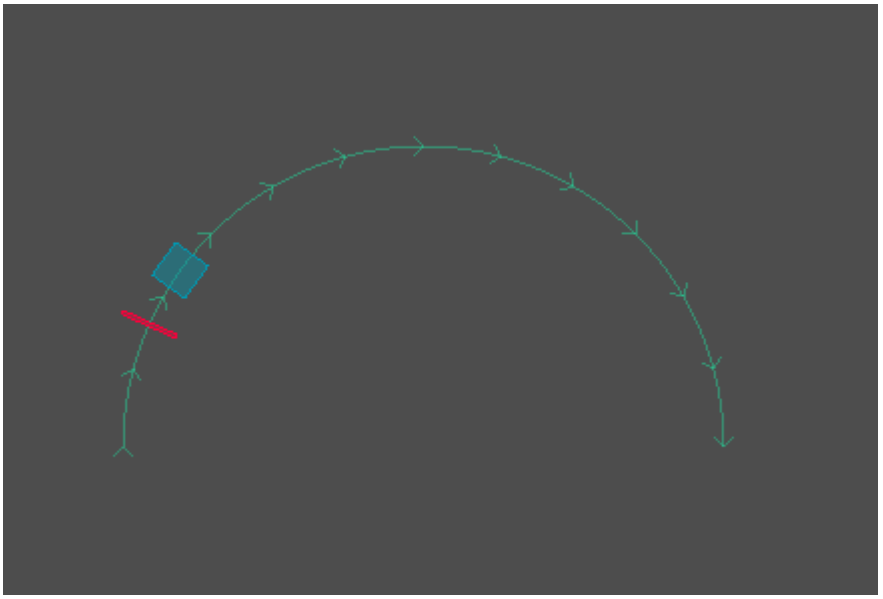


15- To test the scene, ensure In Play is checked (`true`) inside `SkillCheck`.
(To see it working without textures, enable the debug options under Debug settings).
It should look like this when running the scene:

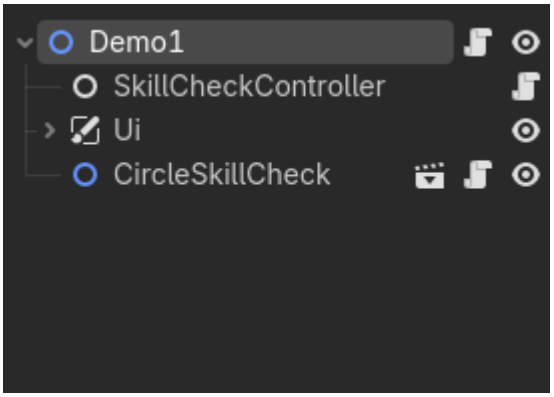




16- It should look like this when running the scene:



17- Explore and study the included Demo, Scene, and Scene-Prefabs files.



18- Read about `SkillCheckController`

SingletonSkillCheck (AutoLoad)

As the name implies, this is a shared script and an Autoload singleton for the resource. It contains the signals emitted during the running minigame. Therefore, you can simply call it in any custom script where you want something to happen when a signal is emitted.

To implement animations and sounds, consider using separate nodes and scripts connected to the Autoload's signals.

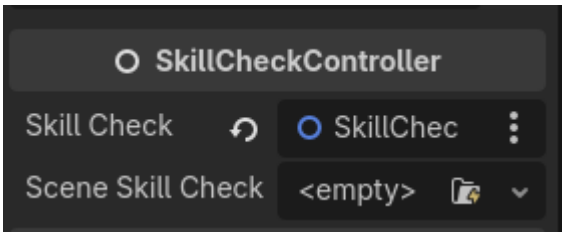
Signals

`signal hit_registered(area: AreaHitZone, type: String, point: float)`
Emitted when a hit occurs within a HitZone area.
It provides (passes as arguments) which area (`AreaHitZone`) was hit, its type (`Anyway`, `Great`, or `Perfect`), and the point value (`float`).
`signal miss`
Emitted when the player misses and fails to hit any hit zone.
`signal end()`
Emitted when the minigame ends.

SkillCheckController

This Node serves as an intermediary to control the Minigame from another Script during gameplay — for example, if you want to present a tutorial or a cutscene/cinematic to the player beforehand.

Inspector



`@export var _skill_check : SkillCheck`
Reference to the Minigame instance.
`@export var _scene_skill_check_model : PackedScene`
Saved scene of the Minigame to be instantiated using the `spawn_my_skill_check(_scene_skill_check_model : PackedScene)` method.

Methods.

_ready()

```gdscript
func _ready() -> void:
	SingletonSkillCheck.connect("hit_registred", hit_process)
	SingletonSkillCheck.connect("hit_registred", func(area : AreaHitZone, type : String, point : float):
		get_hit_zones_count())
	SingletonSkillCheck.connect("miss",miss_process)
	SingletonSkillCheck.connect("end",end_process)
```

play_skill_check():

```gdscript
func play_skill_check():
	if !_skill_check:
		print_debug("Null Skill Check")
		return
	_skill_check._play_or_pause(true)
```

pause_skill_check():

```
func pause_skill_check():
    _skill_check._play_or_pause(false)
```

Methods Connected to `SingletonSkillCheck` AutoLoad Signals

These methods define how the `SkillCheckController` responds when signals are emitted.

```
func hit_process(area : AreaHitZone, type : String, point : float):
    pass
func miss_process():
    pass
func end_process():
    pass
```

spawn_my_skill_check(_scene_skill_check : PackedScene):

```
func spawn_my_skill_check(_scene_skill_check : PackedScene):
    var new_skill_check : SkillCheck = _scene_skill_check_model.instantiate()
    add_child(new_skill_check)
    _skill_check = new_skill_check
```

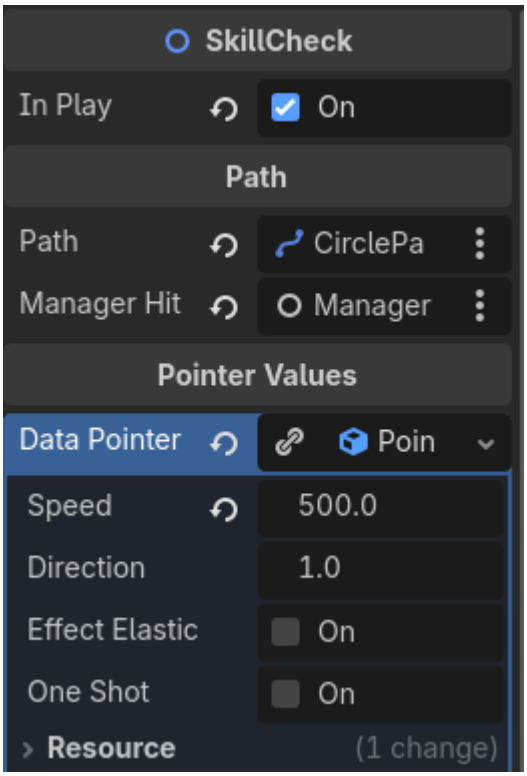
get_hit_zones_count():

```
func get_hit_zones_count():
    _skill_check.get_hit_zone_count()
    pass
```

Skill Check

`SkillCheck` is the class responsible for the logic and runtime execution of the minigame while it is running.

Inspector



In Play: Initial attribute determining whether the minigame is playable immediately after loading.
Path: Reference to the `PathSkillCheck` node used for the path layout and spatial positioning.
Manager Hitzones: Reference to the `ManagerHitZones` node that manages the organizational rules for hitzones.
Data Pointer: Reference to the `ResourcePointer`. Allows modifying the attributes directly from `SkillCheck` in the Inspector that the Pointer will use. *(For more details, see the "Pointer" section).*

Methods

_physics_process()

Checks player input for the core mechanic while the minigame is active.

```gdscript
func _physics_process(_delta: float) -> void:
	#Core gameplay input check during physics execution
	if Input.is_action_just_pressed("ui_accept"):
		if _pointer.get_hittable():
			var current_hit_zone : AreaHitZone = _pointer._current_hit_zone
			SingletonSkillCheck.hit_registred.emit(current_hit_zone, current_hit_zone.get_type(), current_hit_zone.point
		else:
			SingletonSkillCheck.miss.emit()
```

_play_or_pause(play : bool)

Controls play/pause state exclusively for the minigame instance.

```gdscript
func _play_or_pause(play : bool):
	if !play:
		set_physics_process(play)
		_pointer.set_physics_process(play)
	else:
		set_physics_process(play)
		_pointer.set_physics_process(play)
```

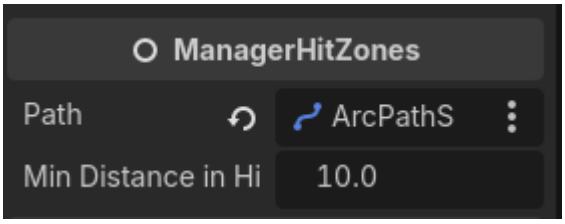
get_hit_zone_count()

Checks if there are remaining HitZones on the `PathSkillCheck`. If none remain, the minigame emits the `end` signal.

```gdscript
func get_hit_zone_count():
	if path.get_hit_zones().size() <=0:
		SingletonSkillCheck.end.emit()
```

Manager HitZones

This component is responsible for organizing the distribution of HitZones along the `PathSkillCheck` (the path that the pointer will follow).



Path: Select the `PathSkillCheck` in the Inspector that this component should organize. Otherwise, the component will not arrange the HitZones.

Min Distance in HitZones: A `float` value representing the minimum distance in pixels between HitZones when there is more than one HitZone present.

Methods

_ready()

```
func _ready() -> void:
    path.connect("new_hit_zones",update)
    var hit_zones : Array[HitZone] = path.get_hit_zones()
    var size_path : float = path.get_size_path()
    organize_hit_zones(hit_zones,size_path, min_distance_in_hit_zones)
```

organize_hit_zones()

```gdscript
func organize_hit_zones(hit_zones: Array[HitZone],size_path: float, min_distance: float = 10.0) -> void:
	var total_zones = hit_zones.size()
	if total_zones == 0:
		return
	if  total_zones == 1:
		var hit_zone = hit_zones[0]
		var min_path : float = 0 + (hit_zone.get_width_texture()/2)
		var max_path : float = (size_path - ( hit_zone.get_width_texture()/2))
		hit_zone.progress = randf_range(min_path,max_path)
		return
	var total_length: float = path.get_size_path()
	var texture_width: float = hit_zones[0].get_width_texture()
	# Minimum required distance between the centers of two HitZones
	var required_separation: float = texture_width + min_distance
	# Validate if the path has enough total space
	if (required_separation * total_zones) > total_length:
		push_warning("The path is too short to fit all HitZones with this minimum clearance!")
	var placed_positions: Array[float] = []
	for i in range(total_zones):
		var position_found: bool = false

		# Attempts to pick a valid random position within attempt limit
		for attempt in range(min_distance):
			# Pick a progress value keeping margins away from path edges
			var candidate_progress: float = randf_range(texture_width / 2.0, total_length - (texture_width / 2.0))
			var valid: bool = true

			# Check separation against already placed zones
			for pos in placed_positions:
				if abs(candidate_progress - pos) < required_separation:
					valid = false
					break

			if valid:
				placed_positions.append(candidate_progress)
				hit_zones[i].progress = candidate_progress
				position_found = true
				break

		# Fallback if random sampling fails to find free space
		if not position_found:
			push_warning("Could not find a valid random position without overlap for HitZone %d." % i)
			var fallback_progress: float = (i * required_separation) + (texture_width / 2.0)
			hit_zones[i].progress = clampf(fallback_progress, 0.0, total_length)
```

update()

```gdscript
func update():
	var hit_zones : Array[HitZone] = path.get_hit_zones()
	var size_path : float = path.get_size_path()
	organize_hit_zones(hit_zones,size_path,min_distance_in_hit_zones)
```

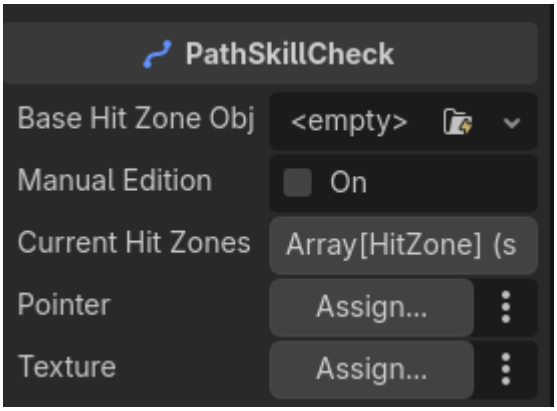
PathSkillCheck

A SkillCheck component that defines the `Path2D` trajectory and provides layout support for HitZones and the Pointer/Selector. It serves as the spatial foundation for trajectory-based minigames, supplying the path geometry and spatial positioning for indicators and hit zones.

Extend this class to implement specific trajectory geometries, such as `CirclePathSkillCheck` or `ArcPathSkillCheck`.

(Note: The overall minigame flow, hit evaluation, and lifecycle logic are managed by the parent `SkillCheck` class).

Inspector



Base HitZone Object: Default scene type (`PackedScene`) used to automatically instantiate a HitZone.
Manual Edition: If `true`, the path must be edited manually; if `false`, it follows rules defined via `rules_build_path()`.
Current Hit Zones: List of HitZones associated with this path.
(Important: Always add your hit zones to this list, even if there is only one).
Pointer: Reference to the Pointer loaded during the `_ready()` method.

Methods

rules_build_path()

If empty, the path must be drawn via the Inspector using the inherited `Path2D` property.

```
func rules_build_path() -> void:
    pass
```

get_size_path() -> float

```
func get_size_path() -> float:
    return curve.get_baked_length()
```

get_hit_zones()-> Array:

```
func get_hit_zones()-> Array:
    return _current_hit_zones
```

get_pointer() -> Pointer

```
func get_pointer():
    return pointer
```

add_hit_zones()

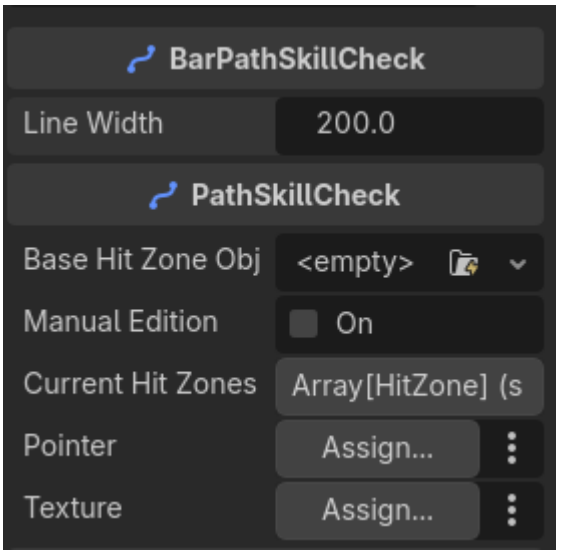
```
func add_hit_zones( amount : int = 1,object_scene : PackedScene = base_hit_zone_Object_Scene ):
    for new_hitzones in amount:
        var new_instantiate : HitZone = object_scene.instantiate()
        _current_hit_zones.append(new_instantiate)
        new_hit_zones.emit()
```

Signal

new_hit_zones

Emitted when a new HitZone is added via the `add_hit_zones()` method

BarPathSkillCheck



Line Width: Accepts a `float` value; draws and displays a line representation directly in the Inspector.

Methods

```gdscript
func rules_build_path():
    build_horizontal_line_path(line_width)

func build_horizontal_line_path(length: float) -> void:
    var new_curve: Curve2D = Curve2D.new()
    var half_length: float = length / 2.0

    # Ponto 1: Extremidade Esquerda
    new_curve.add_point(Vector2(-half_length, 0.0))

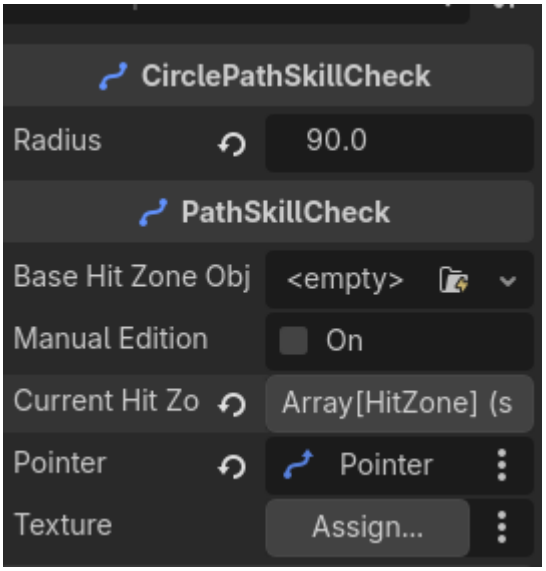
    # Ponto 2: Extremidade Direita
    new_curve.add_point(Vector2(half_length, 0.0))

    curve = new_curve
```

CirclePathSkillCheck

Inherits the features of `PathSkillCheck`, while adding automatic drawing of a circle defined by the **Radius** value in its Inspector. Changing the "Radius" value automatically redraws a new circle in the Inspector.

Inspector



Radius :Accepts a `float` value, drawing and displaying the new circle directly in the Inspector.

Methods

```gdscript
func rules_build_path() -> void:
	create_circle_path()
	pass

## Generates a perfect circular path using Path2D curves.
func create_circle_path():
	var new_curve = Curve2D.new()
	var points = 4 #4 cardinal points
	var handle_length = radius * 0.5522847498 # Constant for optimal Bézier circle approximation

	for i in range(points):
		var angle = i * (PI / 2.0)
		var pos = Vector2(cos(angle), sin(angle)) * radius

		#Control vectors (Handles)
		var dir_in = Vector2(sin(angle), -cos(angle)) * handle_length
		var dir_out = -dir_in

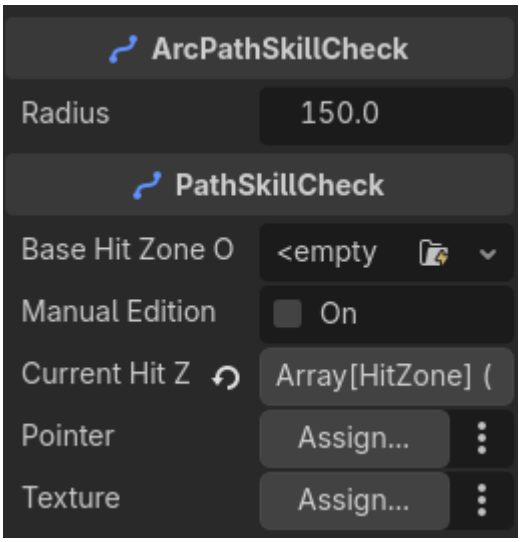
		new_curve.add_point(pos, dir_in, dir_out)

	# Closes the circle by connecting the last point back to the first
	new_curve.add_point(new_curve.get_point_position(0), new_curve.get_point_in(0), new_curve.get_point_out(0))
	curve = new_curve
```

ArcPathSkillCheck

Inherits the features of `PathSkillCheck`, while adding automatic drawing of an Arc defined by the **Radius** value in its Inspector. Changing the "Radius" value automatically redraws a new Arc in the Inspector.

Inspector



Radius: Accepts a `float` value, drawing and displaying the new Arc directly in the Inspector.

Methods

```
func rules_build_path() -> void:
    create_arc()

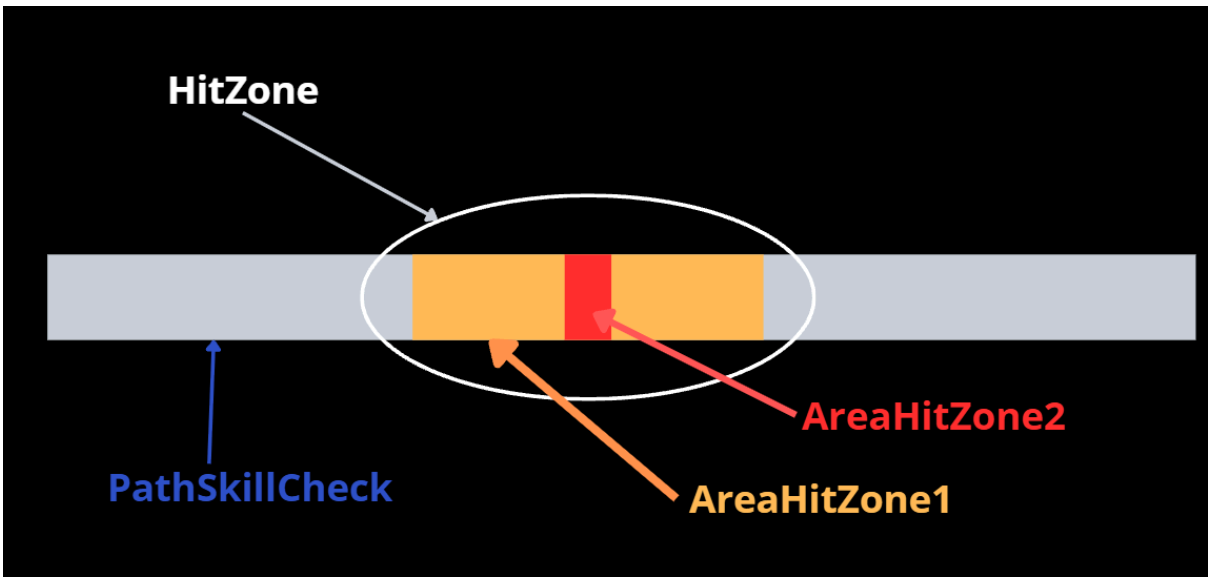
func create_arc():
    var new_curve: Curve2D = Curve2D.new()
    var handle_length: float = radius * 0.5522847

    # Ponto 1: Topo
    new_curve.add_point(Vector2(0, -radius), Vector2.ZERO, Vector2(handle_length, 0))
    # Ponto 2: Vértice Curvo (Direita)
    new_curve.add_point(Vector2(radius, 0), Vector2(0, -handle_length), Vector2(0, handle_length))
    # Ponto 3: Base
    new_curve.add_point(Vector2(0, radius), Vector2(handle_length, 0), Vector2.ZERO)

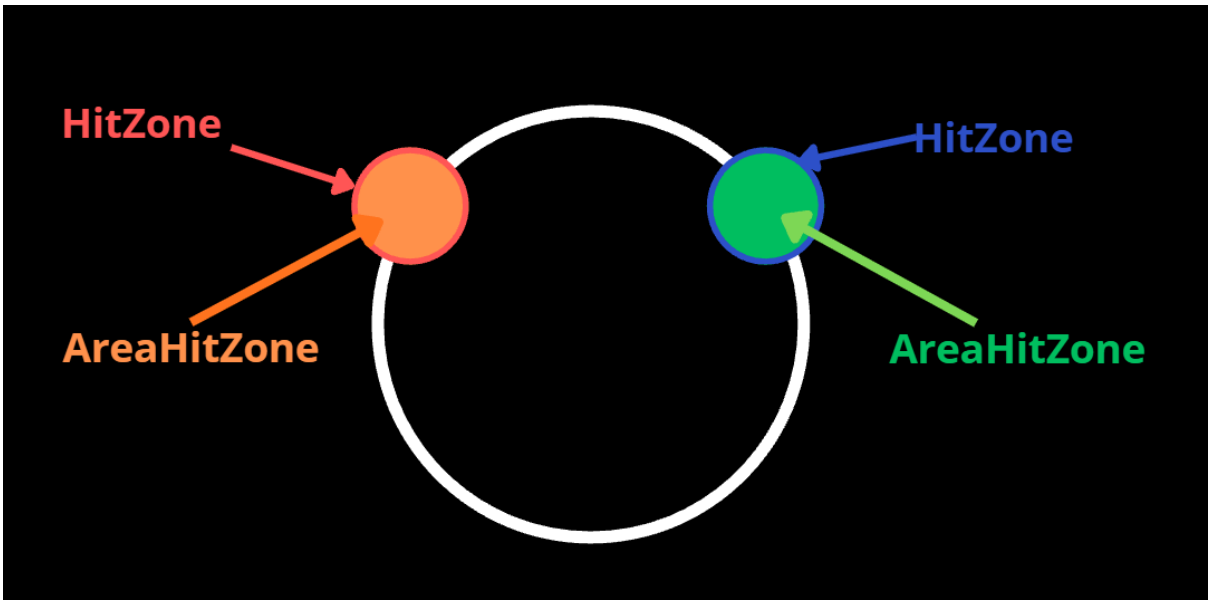
    curve = new_curve
```

HitZone

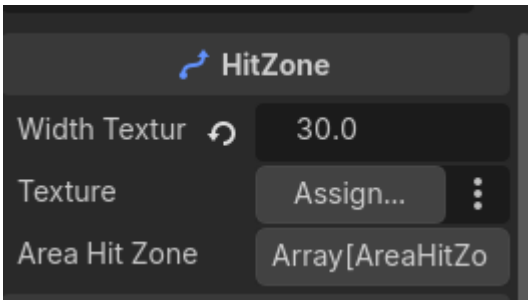
`HitZone` represents a target hit area along a `PathSkillCheck`. A `HitZone` can contain multiple `AreaHitZone` nodes, each with distinct meanings or scoring values — a common pattern in this type of minigame where adjacent areas represent outcomes such as "Perfect" or "Great".In short, a `HitZone` is composed of one or more `AreaHitZone` elements.*Examples:*
1- One `HitZone` containing two `AreaHitZone` elements.



2- Two `HitZone` nodes, each containing one `AreaHitZone`



Inspector



Width Texture: A manual reference for the texture width used in HitZone layout calculations. Due to variations in how shape dimensions are defined across Godot's API, this value

simplifies distribution and positioning calculations.
*(**Note: This property is read-only for layout calculation purposes and does not resize or scale the texture itself. If you have multiple `AreaHitZone` nodes, enter the combined total sum of their widths. Alternatively, height value can be used instead of width if it fits your path better)**.*
Texture: Visual sprite texture used as a background for the HitZone target region. It acts as a backdrop when multiple hit sub-regions exist within a single hit zone; otherwise, it can reference the primary hit zone sprite directly.
Area Hit Zone: Array list responsible for storing the individual areas within the `HitZone`. A single `HitZone` can house multiple sub-areas with different scoring values and hit types.

Methods

get_width_texture() -> float

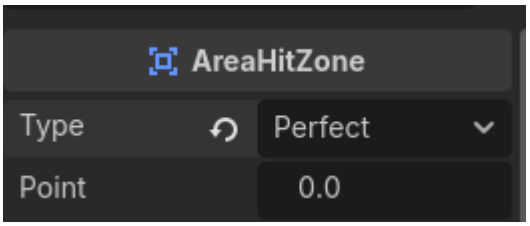
```
func get_width_texture():
    return _width_texture
```

get_hit_area() -> Array

```
func get_hit_area():
    return _area_hit_zone
```

AreaHitZone

Inspector



Type: Choose between "Anyway", "Great", and "Perfect".
Point: A `float` value representing how many points this area is worth.

Methods

get_type()->String:

```
## Returns the string representation of the hit accuracy type
func get_type()-> String:
    var name_type : String = Sucess.keys()[_type]
    return name_type
```

get_size_shape():

Returns the size of the collision shape (specifically tailored for rectangles or squares; other specific methods exist for different shapes).

```
func get_size_shape():
    return _collision_shape.shape.size
```

hittable():

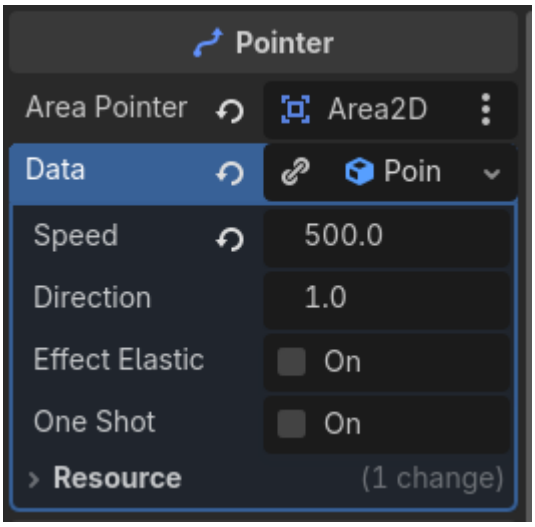
Defines the code block executed when the Area receives a HIT.

```
func hittable():
    queue_free()
```

Pointer

The Pointer is basically the "needle" that travels along the entire path (`PathSkillCheck`). Depending on its position when the user triggers the "Hit" action, it determines whether a `HitZone` was successfully hit or missed.

Inspector



Area Pointer: A standard `Area2D` used exclusively for detecting `HitZone` collisions.
Data: Accepts a `ResourcePointer`. While it can be created directly in the Inspector, using a pre-configured or saved resource is recommended.

- **Speed:** The movement speed along the path.
- **Direction:** The initial travel direction.
- **Effect Elastic:** When it reaches the end of the path, it reverses direction (unlike the standard looping attribute in `PathFollow2D`).
- **One Shot:** Stops moving after traversing the path completely once.

Methods

_ready():

```gdscript
func _ready():
    if data.one_shot && data.effect_elastic:
        data.effect_elastic = false
    loop = !data.effect_elastic
    speed_pointer = data.speed
    pointer_direction = data.direction
    _area_pointer.area_entered.connect(_can_hit)
    _area_pointer.area_exited.connect(not_hit)
```

_physics_process(delta) and move_pointer(delta)

```
func _physics_process(delta: float) -> void:
	move_pointer(delta)

func move_pointer( delta):
	progress += speed_pointer * pointer_direction * delta

	if loop and data.one_shot:
		if progress_ratio >= 0.9:
			progress_ratio = 1.0
			SingletonSkillCheck.end.emit()
			set_physics_process(false)
		return
	else:
		if progress_ratio >= 1.0:
			progress_ratio = 1.0
			pointer_direction = -1.0
		elif progress_ratio <= 0.0:
			progress_ratio = 0.0
			pointer_direction = 1.0
		
```

_is_hit_zone(area : Area2D)

```
func is_hit_zone(area : Area2D) -> bool:
	if area is AreaHitZone:
		return true
	else:
		return false
```

_can_hit(area : Area2D)

```
func _can_hit(area : Area2D):
    var i_can : bool = is_hit_zone(area)
    if !i_can:
        return
    else:
        hittable = true
        _current_hit_zone = area
```

not_hit(area : Area2D)

```
func not_hit(area : Area2D):
    var i_not : bool = is_hit_zone(area)
    if !i_not:
        return
    else:
        hittable = false
        _current_hit_zone = null
```

getters

```
func get_speed_pointer():
    return speed_pointer
func get_hittable():
    return hittable
func get_direction():
    return pointer_direction
```

ResourcePointer

Attributes

- *`@export var speed : float`*
- *`@export var direction : int`*
- *`@export var effect_elastic : bool`*
  Toggles the elastic rebound effect for the pointer's movement.
  - *When enabled, path looping is automatically disabled.*

    - *Must remain disabled on full circular paths.*
- `@export var one_shot : bool`
  When enabled, the minigame ends automatically after the pointer completes a single full traversal.
    - *Cannot be used in conjunction with `effect_elastic`.*

License

MIT License

Copyright (c) 2026 ShakerCTRL

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

Conclusion

This asset was a personal challenge I set for myself to build something meaningful using my knowledge of game engines and software engineering principles. Artificial Intelligence was used solely for specific mathematical calculations and for translating the documentation into English — the entire architecture, structure, and core functionality were designed and developed by me.

I acknowledge that it isn't perfect, and I am aware of its current limitations and installation process. However, I gave it my absolute best, and I plan to refine its architecture in the future or adapt it into a full Godot Plugin.

If you took the time to read through this documentation and explore the framework, **thank you so much!**

Follow on Social Media:

@ShakerCTRL